Parsek — Logistics Routes Design

Design specification for Parsek's automated resource delivery system — covering route creation, dispatch scheduling, resource transfer between unloaded vessels, endpoint resolution, transfer window computation, and round-trip linking across the rewind timeline.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies how looped recordings are extended into logistics routes that physically move resources between vessels.

Version: 0.1 (design phase — no implementation yet) **Prerequisite:** Phase 11 (Resource Snapshots) must be implemented before routes. Phase 10 (Location Context) is already complete. **Out of scope:** Ghost playback engine, recording system, chain structure, game actions recalculation engine. See `parsek-flight-recorder-design.md` and `parsek-game-actions-and-resources-recorder-design.md` for those.

---

1. Introduction

This document specifies how Parsek turns looped recordings into automated supply routes. It covers:

- What makes a recording eligible to become a route (dock + transfer + undock validation)
- How the delivery manifest is computed from what the player actually transferred
- The dispatch cycle: check timing, check capacity, check origin resources, deduct, deliver
- Endpoint resolution: surface proximity fallback vs orbital PID matching
- Transfer window scheduling via synodic period computation
- Round-trip linking for paired one-way routes
- Resource modification on unloaded vessels via ProtoPartResourceSnapshot
- Timeline integration with epoch isolation for revert safety
- Edge cases: destruction, full tanks, competing routes, time warp, reverts

1.1 What happens when the player creates a route

The player flies a cargo mission manually — drives a rover with fuel to a base, or launches a tanker to an orbital station. During the flight, they dock at the destination, transfer resources using KSP's standard UI, and undock. Parsek records the whole thing.

After committing the recording, a "Create Route" button appears. Clicking it creates an automated logistics route. Parsek derives everything from the recording: where the route starts and ends, what resources are delivered, how long transit takes, and how much fuel the origin must supply. The player confirms and optionally adjusts the dispatch interval.

1.2 What happens each cycle

On schedule, the route evaluates whether delivery is possible. It checks the destination (is there room?), the origin (are there enough resources?), and whether the transit window is open. If everything checks out, origin resources are deducted, and after the recorded transit duration elapses, resources appear at the destination. The ghost replays the recorded flight visually during transit.

1.3 What the player sees

Situation	What happens
Create route from recording	System auto-derives origin, destination, delivery manifest, and cost. Player confirms.

Route dispatches on schedule	Origin resources deducted (if non-KSC). Ghost begins replay.
Route delivers after transit	Resources appear in destination vessel tanks. Ghost completes cycle.
Destination tanks full	Cycle skipped. Origin NOT deducted. Ghost loops visually.
Origin runs out of resources	Dispatch delayed until resources available. Route resumes automatically.
Destination destroyed (surface)	Proximity fallback auto-reconnects to rebuilt base at same location.
Destination destroyed (orbital)	Route halted (`EndpointLost`). Player must re-target to new station.
Player reverts past a dispatch	Epoch isolation invalidates dispatch. Origin resources restored.
Two routes linked as round-trip	They alternate: Route A completes → Route B dispatches → B completes → A dispatches.

1.4 Example: fuel delivery rover

```
RECORDING (committed):
  Segment 1: Rover departs KSC runway with 200 LF
  Segment 2: Rover docks at base (pre-dock: 195 LF)
  Segment 3: Player transfers 150 LF to base (pre-undock: 45 LF)
  Segment 4: Rover undocks, drives back to KSC

ROUTE DERIVED:
  Origin:     KSC runway area (free — no deduction)
  Destination: base location (from dock event coordinates)
  Delivery:   150 LF per cycle (195 pre-dock – 45 pre-undock)
  Transit:    recording duration (~10 min)
  Interval:   player-set (minimum = recording duration)

EACH CYCLE:
  UT=0:     Dispatch. Ghost rover starts replay.
  UT=600:   Delivery. 150 LF added to base tanks.
            If base full → 0 LF added, origin not deducted.
```

1.5 Example: Minmus mining supply chain

```
ROUTE A: KSC → Minmus Base (fuel supply)
  Origin:     KSC (free)
  Delivery:   800 LF + 978 Ox per cycle
  Result:     mining base stays fueled for drill + ISRU operation

ROUTE B: Minmus Base → Kerbin Depot (ore delivery)
  Origin:     Minmus base (non-KSC — must have resources)
  Cost:       1200 Ore + 600 LF + 733 Ox per cycle (start manifest)
  Delivery:   1200 Ore per cycle
```

```
  Gate:       dispatch delayed until base has all required resources

CHAIN BEHAVIOR:
  Route A delivers fuel → base mines ore → Route B ships ore to depot.
  If Route A stops → base runs dry → Route B pauses indefinitely.
  If depot tanks full → Route B skips cycles, base accumulates ore.
```

---

2. Design Philosophy

These principles govern every design decision in the logistics system. They are listed here because they inform every section that follows.

2.1 Realism and fidelity

1. **Fly it once, automate it forever.** Routes replicate exactly what the player did during the recording. The delivery amount, transit duration, and fuel cost are all derived from the real flight — not configured abstractly.
2. **Transit takes real time.** The route duration equals the recording duration. A 3-year Eeloo transfer takes 3 years per cycle. A 10-minute rover drive takes 10 minutes. No shortcuts.
3. **Transfer windows are respected.** Inter-body routes dispatch at the synodic period of the origin and destination bodies. Same-body routes can dispatch at any time but not faster than the original recording.

2.2 Resource safety

4. **No infinite resource glitches.** Routes deliver exactly what was transferred during the recording — no more, no less. KSC origins are free (KSP charges funds to build vessels). Non-KSC origins deduct the transport's full start manifest. Recovery funds from the real vessel are one-time.
5. **Don't waste origin resources.** Origin is only deducted if at least one delivery resource can be accepted at the destination. If destination is completely full, the cycle is skipped and origin pays nothing. Per-resource delivery is independent: each resource fills what fits.
6. **Dock + transfer + undock required.** A route can only be created from a recording where the transport docked, transferred resources, AND undocked (freeing the port for the next cycle). No undock = no route.

2.3 Abstraction model

7. **No physical vessel during transit.** Route execution is pure math — deduct at origin, wait, add at destination. The ghost is visual only. No physics, no collisions, no orbit propagation.
8. **Endpoints are locations, not specific vessels.** Surface endpoints use vessel PID as primary match, with 50m coordinate fallback if the vessel is gone. Orbital endpoints use PID only. This survives base rebuilding, vessel replacement, and mod compatibility (transfer tubes, claws, etc.).
9. **The system doesn't produce resources.** Parsek moves resources between locations. Mining, ISRU, and solar power are the player's responsibility. Routes chain naturally: the output of one feeds the input of another.

2.4 Timeline integration

10. **Dispatches and deliveries are timeline events.** Route activity participates in the same ledger and epoch isolation system as game actions. Reverts invalidate dispatches from abandoned timelines. Resources are restored via quicksave.

11. **Routes persist across scenes and save/load.** All route state is serialized in the .sfs. The scheduler runs in all scenes via ParsekScenario.

---

3. Terminology

Route — a separate entity that defines a repeating resource transfer between two locations. Created from a recording that contains a valid dock-transfer-undock sequence. Uses the recording's loop system for timing and ghost visuals.

Endpoint — origin or destination of a route. Defined by body, coordinates, and the vessel PID from the dock event. Surface endpoints fall back to 50m proximity if the PID vessel is gone. Orbital endpoints use PID only.

Delivery manifest — the per-resource amounts transferred per cycle. Computed as pre-dock minus pre-undock transport resources (only decreases).

Cost manifest — the transport vessel's resources at recording start. For non-KSC origins, this is deducted from the origin each cycle (cargo + transit fuel).

Dispatch — the moment a route cycle begins. Origin resources are checked and deducted. A timeline event is created.

Delivery — the moment a route cycle completes (after transit duration). Resources are added to the destination. A timeline event is created.

Round-trip link — a scheduling constraint pairing two one-way routes: "don't dispatch me until my partner completes."

---

4. Data Model

4.1 ResourceAmount

```
internal struct ResourceAmount
{
    public double amount;
    public double maxAmount;
}
```

4.2 RouteEndpoint

```
internal struct RouteEndpoint
{
    public string bodyName;        // e.g., "Kerbin", "Mun"
    public double latitude;        // from dock event trajectory point
    public double longitude;
    public double altitude;
    public uint vesselPid;         // PID of vessel docked to during recording
```

```
    public bool isOrbital;         // true = no surface proximity fallback
}
```

4.3 RouteStatus

```
internal enum RouteStatus
{
    Active,             // dispatching on schedule
    InTransit,          // dispatched, waiting for transit duration to elapse
    WaitingForResources,// origin exists but lacks resources — delayed
    DestinationFull,    // destination can't accept delivery — skipping cycles
    EndpointLost,       // destination/origin vessel gone (orbital PID miss or no surface
vessels)
    Paused              // player manually paused
}
```

4.4 Route

```
internal class Route
{
    // Identity
    public string Id;                    // unique route ID (GUID)
    public string RecordingId;           // source recording ID
    public string Name;                  // player-visible name (editable)

    // Endpoints
    public RouteEndpoint Origin;
    public RouteEndpoint Destination;
    public bool IsKscOrigin;             // true = no origin deduction

    // Resource transfer
    public Dictionary<string, double> DeliveryManifest;  // per-resource delivery amounts
    public Dictionary<string, double> CostManifest;      // per-resource origin cost (start
manifest)

    // Timing
    public double TransitDuration;       // seconds (= recording duration)
    public double DispatchInterval;      // seconds between cycle starts
    public double NextDispatchUT;        // UT of next scheduled dispatch
    public double? PendingDeliveryUT;    // UT when in-transit delivery arrives (null if not
in transit)
    public Dictionary<string, double> PendingDeliveryAmounts; // actual amounts (computed at
dispatch, survives save/load)

    // Linking
    public string LinkedRouteId;         // paired route for round-trip (null if standalone)

    // State
    public RouteStatus Status;
```

```
    public int CompletedCycles;          // total successful deliveries
    public int SkippedCycles;            // cycles skipped (destination full, origin empty)
}
```

4.5 Serialization format

```
ROUTE
{
    id = <guid>
    recordingId = <recording-guid>
    name = Mun Fuel Run

    ORIGIN
    {
        bodyName = Kerbin
        latitude = -0.0972
        longitude = -74.5577
        altitude = 75.2
        vesselPid = 12345
        isOrbital = False
    }
    DESTINATION
    {
        bodyName = Mun
        latitude = 3.2001
        longitude = -45.1234
        altitude = 612.5
        vesselPid = 67890
        isOrbital = False
    }

    isKscOrigin = True
    transitDuration = 12345.6
    dispatchInterval = 43200.0
    nextDispatchUT = 55000.0
    pendingDeliveryUT = -1
    linkedRouteId =
    status = Active
    completedCycles = 5
    skippedCycles = 1

    DELIVERY_MANIFEST
    {
        LiquidFuel = 150.0
        Oxidizer = 183.3
    }
    COST_MANIFEST
    {
        LiquidFuel = 200.0
        Oxidizer = 244.0
    }
```

```
    PENDING_DELIVERY
    {
        // present only when Status == InTransit
        LiquidFuel = 100.0
        Oxidizer = 0.0
    }
}
```

Routes are stored in ParsekScenario's save data alongside recordings and game actions. Additive — saves without routes load fine. Routes reference recordings by ID but are independent entities.

4.6 Recording extensions (Phase 11)

Two new optional fields on Recording:

```
public Dictionary<string, ResourceAmount> StartResources;  // manifest at recording start
public Dictionary<string, ResourceAmount> EndResources;     // manifest at recording end
```

Serialized as:

```
RESOURCE_MANIFEST
{
    RESOURCE { name = LiquidFuel  startAmount = 3600  startMax = 3600  endAmount = 200  endMax
= 3600 }
    RESOURCE { name = Oxidizer    startAmount = 4400  startMax = 4400  endAmount = 244  endMax
= 4400 }
}
```

Additive — missing node = no data. No format version bump.

---

5. Route Creation

5.1 Validation

A recording qualifies as a route if and only if:

1. At least one dock event exists (PartEventType.Docked boundary in chain)
2. At least one undock event exists AFTER the dock (PartEventType.Undocked boundary)
3. At least one resource decreased on the transport between the dock and undock snapshots

If validation fails: "Create Route" button is absent or greyed out with tooltip explaining what's missing (e.g., "Transport must undock from destination to enable route — docking port needs to be free for the next cycle").

5.2 Route derivation

All values are automatically derived from the recording:

- **Origin** = recording start location (body + coordinates from first trajectory point)
- **Destination** = dock event location (body + coordinates from dock boundary trajectory point)
- **Origin vessel PID** = vessel PID at recording start
- **Destination vessel PID** = PID of vessel docked to (from dock event metadata)
- **IsKscOrigin** = true if origin body is Kerbin and coordinates are near a launch site

- **DeliveryManifest** = ExtractResourceManifest(pre-dock) minus ExtractResourceManifest(pre-undock), per resource, only positive deltas
- **CostManifest** = ExtractResourceManifest(recording start snapshot)
- **TransitDuration** = recording EndUT minus StartUT
- **DispatchInterval** = TransitDuration (default; player can increase). For inter-body routes: defaults to synodic period of origin and destination bodies.

5.3 Player confirmation

Route configuration panel shows derived values. Player can edit name, dispatch interval, and enable/disable. On confirm, route is created and scheduling begins.

5.4 Multiple dock/undock in one recording

v1: only the LAST dock-transfer-undock sequence counts. The destination is the last docking target. Delivery is from the last dock/undock pair. Multi-stop routes deferred to v2.

---

6. Dispatch and Delivery

6.1 Dispatch evaluation

Trigger: The route scheduler runs each physics frame (or once per second during warp) in all scenes via ParsekScenario.

For each route with Status in {Active, WaitingForResources, DestinationFull} and `NextDispatchUT <= currentUT`:

Routes with Status InTransit, Paused, or EndpointLost are excluded from dispatch evaluation.

Step 1: Check round-trip link. If `LinkedRouteId` is set and the linked route has `Status == InTransit`, skip — wait for partner to complete.

Step 2: Check destination. Find vessels at destination endpoint (§7). If NO vessels found at all → set `Status = EndpointLost`, skip cycle. If vessels found but zero capacity for all delivery resources → set `Status = DestinationFull`, increment `SkippedCycles`, advance `NextDispatchUT`. If capacity available → compute per-resource independent delivery: for each resource, deliver `min(deliveryAmount, destinationCapacity)`. Store in `PendingDeliveryAmounts`.

Step 3: Check origin (if non-KSC). Find vessels at origin endpoint (§7). If no vessels found at all → set `Status = EndpointLost`, skip cycle. If vessels exist but insufficient resources for CostManifest → set `Status = WaitingForResources`, do NOT advance `NextDispatchUT` (re-check next frame). Note: only one dispatch per route per evaluation cycle — no catch-up storm.

Step 4: Dispatch. Deduct full CostManifest from origin (transit fuel burned regardless of partial delivery). Store actual delivery amounts in `PendingDeliveryAmounts`. Set `PendingDeliveryUT = currentUT + TransitDuration`. Set `Status = InTransit`. Create ROUTE_DISPATCHED timeline event. Advance `NextDispatchUT`.

6.2 Delivery execution

Trigger: Route has `PendingDeliveryUT <= currentUT`.

1. **Find destination vessels** at endpoint (§7). If NO vessels found → clear `PendingDeliveryUT` and `PendingDeliveryAmounts`, set `Status = EndpointLost`, log warning. Resources already deducted from origin are lost (transit cost of a failed delivery). This prevents InTransit deadlock.

2. **For each resource in `PendingDeliveryAmounts`:** distribute across destination vessel tanks, clamped to current `maxAmount`. For unloaded vessels: modify `ProtoPartResourceSnapshot.amount` directly, respect `flowState`. For loaded vessels: use `Part.RequestResource()`.
3. **Create ROUTE_DELIVERED timeline event.** Record amounts actually delivered.
4. **Clear `PendingDeliveryUT` and `PendingDeliveryAmounts`.** Set `Status = Active`. Increment `CompletedCycles`.
5. **If linked route exists:** the partner route's linked-wait condition is now cleared.

6.3 Per-resource independent delivery

Each delivery resource is evaluated independently:

```
For each resource in DeliveryManifest:
    capacity = sum of (maxAmount - amount) across all destination tanks for this resource
    deliver = min(manifest amount, capacity)
    PendingDeliveryAmounts[resource] = deliver
```

Origin cost: deduct full CostManifest if ANY delivery resource has a non-zero amount. Zero deduction only if total delivery is zero across all resources.

6.4 Pause, unpause, and re-target

Pause: Player clicks Pause in route UI → `Status = Paused`. Route excluded from dispatch evaluation. Ghost may continue looping visually (existing loop system), but no resource operations occur.

Unpause: Player clicks Resume → `Status = Active`. Route re-enters dispatch evaluation on next scheduler tick. `NextDispatchUT` is recalculated if stale (advanced to next valid dispatch time from currentUT).

Re-target (EndpointLost recovery): Player selects a new destination vessel in the route UI → endpoint coordinates and vesselPid updated, `Status` transitions from `EndpointLost` to `Active`. Same mechanism for origin re-targeting on non-KSC routes.

---

7. Endpoint Resolution

7.1 Algorithm

```
ResolveEndpointVessels(endpoint):
    1. Find vessel by endpoint.vesselPid in FlightGlobals.Vessels
       → if found: return [vessel]
    2. If not found AND endpoint.isOrbital:
       → return [] (orbital endpoints have no fallback)
    3. If not found AND endpoint is surface:
       → scan FlightGlobals.Vessels for all vessels within 50m
         of (body, lat, lon, alt)
       → return matches with available tank capacity
```

7.2 Surface vs orbital behavior

- **Surface endpoints:** PID primary, 50m proximity fallback. Handles base rebuilding — a new vessel at the same spot auto-becomes the endpoint.
- **Orbital endpoints:** PID only. Orbital coordinates change every second, so proximity fallback does not work. If an orbital station is destroyed and rebuilt, the player must re-target the route.

7.3 Loaded vs unloaded vessels

If the endpoint vessel is loaded (player is within physics range), use `Part.RequestResource()` for resource operations. If unloaded, use `ProtoPartResourceSnapshot.amount` directly. Both paths apply to origin (deduction) and destination (delivery).

---

8. Transfer Window Scheduling

8.1 Synodic period computation

```
SynodicPeriod(originBody, destBody):
    if originBody == destBody:
        return 0  // same body, no transfer window
    // Walk up to common parent
    a = originBody, b = destBody
    while a.referenceBody != b.referenceBody:
        if a hierarchy depth > b: a = a.referenceBody
        else: b = b.referenceBody
        if a is Sun or b is Sun: return 0
    // a and b now orbit the same parent
    T1 = a.orbit.period, T2 = b.orbit.period
    if T1 == T2: return 0
    return abs(1 / (1/T1 - 1/T2))
```

Handles cross-system routes: Mun→Laythe walks up to Kerbin/Jool orbiting Sun. Guards against Sun-orbiting bodies and equal-period edge cases.

8.2 Dispatch interval rules

- **Same-body routes:** Player-set interval. Minimum = recording duration.
- **Inter-body routes:** Default = synodic period. Player can override.
- **Gravity assist routes:** Two-body synodic approximation (intermediate flybys not tracked). Player can fine-tune.

---

9. Round-Trip Linking

9.1 How round trips work

A round trip is two separate one-way routes, each from its own recording:

1. Player flies tanker KSC → Station. Parsek spawns real tanker at station after first playback.
2. Player flies spawned tanker Station → KSC. Second recording created.
3. Both recordings become routes. Player links them as a round-trip pair.

9.2 Scheduling constraint

Linked routes alternate: **don't dispatch me until my partner completes.**

```
Route A completes → Route B dispatches → Route B completes → Route A dispatches → ...
```

9.3 Implementation

Player selects two routes in the UI and clicks "Link as Round Trip." Sets `LinkedRouteId` on both. The dispatch evaluation (§6.1, step 1) checks whether the partner is `InTransit`. Unlinking clears `LinkedRouteId` on both. Pausing a partner does NOT block the other (linked-wait only applies to `InTransit`).

---

10. Edge Cases

10.1 Destination destroyed, surface base

Scenario: Mun base destroyed. Player rebuilds at same spot. **Behavior:** PID match fails. Surface proximity fallback finds new vessel within 50m. Route auto-reconnects. **v1 limitation:** If rebuilt >50m from original, player must create a new route.

10.2 Destination destroyed, orbital station

Scenario: Kerbin station destroyed. Player rebuilds. **Behavior:** PID match fails. No proximity fallback. `Status = EndpointLost`. Player must re-target.

10.3 Origin destroyed or recovered (non-KSC)

Scenario: Minmus base recovered for funds. **Behavior:** No vessels at origin → `Status = EndpointLost`. Vessels exist but empty → `Status = WaitingForResources`. Route persists. Resumes when resources appear. Surface origins have proximity fallback. KSC origins skip this check.

10.4 Destination tanks full

Scenario: Route delivers 200 LF. Base has 200/200 LF. **Behavior:** `Status = DestinationFull`. Ghost loops visually. Origin NOT deducted. Resumes when player uses fuel.

10.5 Destination partially full

Scenario: Base has room for 100 LF, full on Ox. Delivery: 150 LF + 183 Ox. **Behavior:** Per-resource independent: deliver 100 LF, 0 Ox. Origin pays full CostManifest (transit fuel was burned).

10.6 Player reverts past a dispatch

Scenario: Route dispatched at UT=50000. Player reverts to UT=49000. **Behavior:** Timeline events invalidated by epoch isolation. Origin resources restored via quicksave. Route state restored from .sfs.

10.7 Time warp past multiple cycles

Scenario: Three cycles due at UT=50000, 50500, 51000. **Behavior:** All processed sequentially. Each dispatch checks origin independently. First may deplete origin, blocking subsequent.

10.8 Transport still docked at recording end

Scenario: Player forgets to undock. **Behavior:** Validation fails. "Create Route" absent. Tooltip: "Transport must undock from destination."

10.9 No resources transferred during docking

Scenario: Player docks and undocks without transferring. **Behavior:** Validation fails. Tooltip: "No resource transfer detected during docking."

10.10 Multiple dock/undock in one recording

Scenario: Recording has dock→transfer→undock→dock→transfer→undock. **Behavior:** v1 — last dock-transfer-undock pair counts. Future: multi-stop routes.

10.11 Route dispatch while player is at destination

Scenario: Player at Mun base when delivery arrives. **Behavior:** Destination loaded. Uses `Part.RequestResource()`. Resources appear in real-time.

10.12 Competing routes at same origin

Scenario: Two routes share Minmus base. Base has enough for one, not both. **Behavior:** v1 — FIFO by NextDispatchUT. Future: player-configurable priority.

10.13 Linked route partner paused

Scenario: Route A and B linked. Player pauses B. **Behavior:** A dispatches on its own schedule. B resumes from its next scheduled dispatch when unpaused.

10.14 Recording deleted

Scenario: Source recording for a route is deleted. **Behavior:** Route orphaned — resource transfers continue, ghost replay absent. "No ghost" indicator in UI.

10.15 Save/load round-trip

Scenario: Save, load. **Behavior:** All Route fields serialized in ParsekScenario OnSave/OnLoad. State restored exactly.

---

11. v1 Limitations

- **Capacity changes during transit:** Delivery clamps to `maxAmount` at delivery time. Excess silently lost. Origin was already deducted at dispatch.
- **Zero transit duration:** Dispatch and delivery may process in same frame. Acceptable.
- **EC-only delivery:** Electric charge fluctuates rapidly. Status may flicker between Active and DestinationFull.
- **Resource not on destination:** If delivery manifest includes a resource the destination has no tanks for, that resource is silently skipped.
- **Origin loaded vs unloaded:** Same loaded/unloaded distinction as delivery.
- **Scene handling:** Route scheduler runs in all scenes via ParsekScenario. `FlightGlobals.Vessels` available for endpoint resolution.
- **Revert mechanism:** Route state serialized in .sfs. Quicksave load restores Route ConfigNode. Timeline events use epoch isolation.

---

12. What Doesn't Change

- **Recording system** — recordings unchanged. Resource manifests are additive metadata.
- **Loop system** — timing, ghost replay, cycle events all work as today. Routes add a delivery hook.
- **Ghost playback engine** — no changes to GhostPlaybackEngine, IPlaybackTrajectory, IGhostPositioner.
- **Chain system** — chain segments, dock/undock boundaries, snapshots all unchanged.
- **Merge dialog** — route creation happens after commit, not during merge.
- **Crew reservation** — deferred. Routes don't reserve crew in v1.
- **Game actions system** — route events are new event types in existing ledger. No changes to recalculation engine.
- **Map markers** — deferred. No map view integration in v1.

---

13. Backward Compatibility

- **Saves without routes:** Load fine. ROUTE ConfigNode absent.
- **Saves without resource manifests:** Load fine. Route creation unavailable for old recordings (no manifest data).
- **Old recordings:** Cannot become routes. Player must re-fly to create a recording with manifests.
- **Format:** All new data additive. No version bump. Missing nodes = no data.

---

14. Diagnostic Logging

14.1 Route creation

- `[Parsek][INFO][Route] Route created: id={id} name={name} recording={recId} origin={body}({lat},{lon}) dest={body}({lat},{lon}) delivery={manifest} cost={manifest}`
- `[Parsek][INFO][Route] Route validation failed for recording {recId}: {reason}`

14.2 Dispatch evaluation

- `[Parsek][VERBOSE][Route] Dispatch check: route={name} nextUT={ut} currentUT={ut} status={status}`
- `[Parsek][INFO][Route] Dispatch: route={name} cycle={n} deducted={amounts} from origin at {body}({lat},{lon})`
- `[Parsek][INFO][Route] Dispatch delayed: route={name} — origin missing {resource}={needed} (available={have})`
- `[Parsek][INFO][Route] Dispatch skipped: route={name} — destination full (capacity={amounts})`
- `[Parsek][INFO][Route] Dispatch skipped: route={name} — linked route {linkedId} in transit`

14.3 Delivery

- `[Parsek][INFO][Route] Delivery: route={name} cycle={n} delivered={amounts} to {count} vessel(s) at {body}({lat},{lon})`
- `[Parsek][INFO][Route] Partial delivery: route={name} — {resource} delivered {actual}/{requested}`
- `[Parsek][WARN][Route] Delivery failed: route={name} — no vessels at destination`

14.4 Endpoint resolution

- `[Parsek][VERBOSE][Route] Endpoint resolve: {type} pid={pid} found={bool} fallback={proximity/none} vessels={count}`

14.5 State transitions

- `[Parsek][INFO][Route] Status change: route={name} {old} → {new}`

14.6 Timeline events

- `[Parsek][INFO][Route] Timeline event: ROUTE_DISPATCHED route={name} ut={ut}`
- `[Parsek][INFO][Route] Timeline event: ROUTE_DELIVERED route={name} ut={ut} amounts={amounts}`

---

15. Test Plan

15.1 Unit tests (pure logic, no Unity)

ExtractResourceManifest

- Empty ConfigNode → empty manifest. *Catches: null deref on missing PART nodes.*
- Single part, one resource → manifest with one entry. *Catches: parsing errors.*
- Multi-part, same resource → amounts summed. *Catches: overwrite instead of accumulate.*
- Multiple resource types → all present. *Catches: single-resource assumption.*
- Missing RESOURCE node → skipped gracefully. *Catches: null deref.*
- Zero-amount resource → included (maxAmount matters). *Catches: filtering out zeros.*

ComputeDeliveryManifest

- Normal transfer: pre-dock 200 LF, pre-undock 50 LF → delivery 150 LF. *Catches: wrong snapshot pair.*
- Resource increased (EC): pre-dock 50, pre-undock 180 → delivery 0. *Catches: including increases.*
- Mixed: some decrease, some increase → only decreases. *Catches: all-or-nothing.*
- No decreases → empty manifest (validation rejects). *Catches: false positive.*

Route validation

- Dock + transfer + undock → valid. *Catches: false rejection.*
- Dock + undock, no transfer → invalid. *Catches: false acceptance.*
- Dock + transfer, no undock → invalid. *Catches: missing undock check.*
- No dock → invalid. *Catches: missing dock check.*

Dispatch evaluation

- KSC origin, capacity available → dispatch, no deduction. *Catches: deducting from KSC.*
- Non-KSC, sufficient → dispatch, deducted. *Catches: skipping deduction.*
- Non-KSC, insufficient → delayed. *Catches: dispatching without resources.*
- Destination full → skipped, origin NOT deducted. *Catches: wasted deduction.*
- Partial capacity → per-resource independent, full cost. *Catches: coupled delivery.*
- Linked partner in transit → skipped. *Catches: ignoring link.*

Synodic period

- Kerbin-Mun → ~6.4 days. *Catches: formula error.*
- Mun-Laythe → walks to Kerbin/Jool. *Catches: cross-system hierarchy.*
- Same body → 0. *Catches: division by zero.*
- Sun-orbiting → 0. *Catches: missing guard.*
- Equal periods → 0. *Catches: T1==T2 division.*

Endpoint resolution

- PID exists → found. *Catches: skipping PID.*
- PID gone, surface, vessel within 50m → fallback. *Catches: missing fallback.*
- PID gone, surface, nothing nearby → empty. *Catches: false match.*
- PID gone, orbital → empty. *Catches: orbital fallback.*

15.2 Log assertion tests

- Route creation logs manifest and endpoints. *Catches: silent creation.*
- Dispatch delayed logs missing resource and amounts. *Catches: silent delay.*
- Delivery logs amounts per vessel. *Catches: silent delivery.*
- Status change logs old→new. *Catches: silent transition.*
- Partial delivery logs reason. *Catches: silent partial.*

15.3 Serialization round-trip tests

- Route serialize → deserialize → all fields match. *Catches: missing field.*
- ResourceManifest round-trip with full precision. *Catches: locale formatting.*
- Null LinkedRouteId survives round-trip. *Catches: empty vs null.*
- In-transit route with PendingDeliveryUT survives. *Catches: transit state lost.*

15.4 Integration tests (synthetic recordings)

- Recording with dock+transfer+undock → validation passes. *Catches: chain structure mismatch.*
- Recording without undock → validation rejects. *Catches: missing chain event check.*
- Inject route into save → loads correctly. *Catches: ParsekScenario integration.*

---

16. Open Questions (deferred to v2)

- **Multi-stop routes:** Recording with multiple dock-transfer-undock sequences. v1 uses only the last pair.
- **Crewed routes:** Crew reservation for route dispatches. v1 ignores crew.
- **Map view integration:** Route lines on the map. Deferred.
- **Dispatch priority for competing routes:** v1 uses FIFO by NextDispatchUT.

Appendix A: Gameplay Scenarios (from Step 2)

A.1 Fuel Delivery Rover

Base with empty fuel tank near KSC. Rover drives to base, docks, transfers 150 LF, undocks, drives back. Route: 150 LF per cycle, KSC origin (free), interval = recording duration.

A.2 Orbital Monoprop Resupply

Kerbin station at 100km. Capsule from KSC: launch, dock, transfer 650 MP, undock, deorbit. Ghost visible during launch and station approach (RELATIVE frame). Transit invisible.

A.3 Minmus Ore Delivery (chained routes)

Route A: KSC → Minmus base (fuel). Route B: Minmus base → Kerbin depot (ore). B is gated by resource availability at Minmus. A feeds B.

A.4 Eeloo Supply Run (interplanetary)

Dispatch at Kerbin-Eeloo synodic period (~1.9 years). Gravity assists use same two-body approximation. Player can override.

A.5 Failure Cases

Destination destroyed (surface: proximity reconnects; orbital: route pauses). Origin empty (delayed). Destination full (skipped, no deduction). Revert (epoch isolation). Time warp (sequential processing). Transport still docked (validation rejects).

Appendix B: Reference Documents

- `docs/dev/research/logistics-network-design.md` — logistics network research
- `docs/dev/research/loop-playback-and-logistics.md` — loop mechanics and orbital drift

- `docs/dev/research/resource-snapshots-preparation.md` — infrastructure analysis and unloaded vessel modification
- `docs/mods-references/Kerbalism-resource-system-analysis.md` — background resource processing patterns
- `docs/roadmap.md` — Phase 11, 11.5, 12